

Технічне завдання: Інтеграція квестової системи на бекенді (C# ASP.NET Core)

Огляд задачі

Мета завдання — реалізувати безпечну серверну логіку для системи квестів. Нам потрібно захистити ігровий контент від несанкціонованого скачування (датамайнінгу) та запобігти чітерству з боку клієнта (накрутка досвіду за виконання квестів).

Принцип роботи:

1. Конфігурації квестів (тексти, цілі, нагороди) зберігаються на сервері як JSON-файли у закритій папці `Quests` (не в `wwwroot`).
2. В базі даних зберігається тільки прогрес гравця (який квест зараз виконується, скільки кроків пройдено, які квести вже завершено).
3. Гравець може запросити конфігурацію тільки свого **поточного активного квеста**. Сервер бере відповідний JSON з диска, підставляє туди поточний прогрес гравця з БД та відправляє клієнту.
4. Досвід за квести нараховує виключно сервер після перевірки виконання цілей.

1. Зміни в Базі Даних (Entity Framework Core)

Необхідно створити сутність для відстеження прогресу квестів користувачів.

Модель `UserQuest.cs` (Папка `Models`)

Створіть модель для таблиці зв'язку між гравцем та квестом:

```
using System.ComponentModel.DataAnnotations;

namespace NMTales.Backend.Models;

public class UserQuest
{
    public int Id { get; set; }

    [Required]
    public int UserId { get; set; }

    [Required]
    public string QuestId { get; set; } = string.Empty; // e.g., "quest_1"

    [Required]
    public int CurrentAmount { get; set; }

    [Required]
    public bool IsCompleted { get; set; }
}
```

Налаштування Context (ApplicationContext.cs в папці Data)

Додайте нову таблицю до контексту БД:

```
public DbSet<UserQuest> UserQuests { get; set; }
```

Після цього обов'язково створіть та застосуйте міграцію (dotnet ef migrations add AddUserQuests & dotnet ef database update).

2. Збереження конфігурацій квестів на сервері

Створіть у корені проекту бекенду папку **Quests**. В середині неї створіть підпапку з ідентифікатором NPC (наприклад, npc_test). У цю папку ми будемо класти файли конфігурацій квестів.

Референс структури JSON (Quests/npc_test/quest_1.json):

```
{
  "id": "quest_1",
  "title": "Перший крок",
  "description": "Поговоріть з Тарасом на локації.",
  "rewards": {
    "xp": 100
  },
  "objective": {
    "type": "talk_npc",
    "target": "npc_test",
    "required_amount": 1,
    "current_amount": 0
  }
}
```

3. Контролер QuestController.cs (Папка Controllers)

Необхідно створити новий API-контролер QuestController, захищений атрибутом [Authorize]. Він повинен містити три е Third ендпоінти.

Рекомендований шаблон контролера:

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using NMTales.Backend.Data;
using NMTales.Backend.Models;
using System.IO;
using System.Text.Json;
using System.Text.Json.Nodes;

namespace NMTales.Backend.Controllers;

[ApiController]
[Route("api/[controller]")]
[Authorize]
public class QuestController : ControllerBase
```

```

{
    private readonly ApplicationDbContext _context;
    private readonly IWebHostEnvironment _env;

    public QuestController(ApplicationDbContext context, IWebHostEnvironment
env)
    {
        _context = context;
        _env = env;
    }

    /// <summary>
    /// Отримати дані поточного активного квесту з інтегрованим прогресом
гравця.
    /// </summary>
    [HttpGet("active")]
    public async Task<IActionResult> GetActiveQuest()
    {
        // 1. Отримуємо UserId з Claims JWT-токена
        var userIdValue =
User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;
        if (!int.TryParse(userIdValue, out var userId)) return Unauthorized();

        // 2. Шукаємо активний квест користувача в БД
        var userQuest = await _context.UserQuests
            .FirstOrDefaultAsync(uq => uq.UserId == userId && !uq.IsCompleted);

        if (userQuest == null)
        {
            return NoContent(); // Активного квесту немає
        }

        // 3. Зчитуємо конфігурацію квесту з файлу
        // Оскільки квест належить конкретному NPC, шлях має включати npc_id.
        // Для простоти поки що шукаємо файл за шаблоном
"Quests/{npc_id}/{quest_id}.json".
        // NPC ID можна витягти з БД або структурувати файли інакше. Для початку
шукаємо в загальній папці.
        var filePath = Path.Combine(_env.ContentRootPath, "Quests", "npc_test",
"${userQuest.QuestId}.json");
        if (!System.IO.File.Exists(filePath))
        {
            return NotFound("Quest file not found on server.");
        }

        // 4. Парсимо JSON та підставляємо актуальний прогрес із БД
        var jsonString = await System.IO.File.ReadAllTextAsync(filePath);
        var jsonNode = JsonNode.Parse(jsonString);

        if (jsonNode != null && jsonNode["objective"] is JsonObject
objectiveNode)
        {
            objectiveNode["current_amount"] = userQuest.CurrentAmount;
        }

        return Ok(jsonNode);
    }

    /// <summary>
    /// Прийняти новий квест.
    /// </summary>
    [HttpPost("accept/{npcId}/{questId}")]
    public async Task<IActionResult> AcceptQuest(string npcId, string questId)
    {

```

```

        var userIdValue =
User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;
        if (!int.TryParse(userIdValue, out var userId)) return Unauthorized();

        // Перевіряємо, чи немає вже активного квесту
        var hasActive = await _context.UserQuests.AnyAsync(uq => uq.UserId ==
userId && !uq.IsCompleted);
        if (hasActive)
        {
            return BadRequest("You already have an active quest.");
        }

        // Перевіряємо, чи існує такий квест на сервері
        var filePath = Path.Combine(_env.ContentRootPath, "Quests", npcId,
${questId}.json");
        if (!System.IO.File.Exists(filePath))
        {
            return NotFound("Requested quest was not found.");
        }

        // Створюємо запис прогресу
        var newUserQuest = new UserQuest
        {
            UserId = userId,
            QuestId = questId,
            CurrentAmount = 0,
            IsCompleted = false
        };

        _context.UserQuests.Add(newUserQuest);
        await _context.SaveChangesAsync();

        return Ok(new { message = "Quest accepted successfully." });
    }

    /// <summary>
    /// Сдати поточний активний квест та отримати досвід.
    /// </summary>
    [HttpPost("complete")]
    public async Task<IActionResult> CompleteQuest()
    {
        var userIdValue =
User.FindFirst(System.Security.Claims.ClaimTypes.NameIdentifier)?.Value;
        if (!int.TryParse(userIdValue, out var userId)) return Unauthorized();

        // 1. Знаходимо активний квест
        var userQuest = await _context.UserQuests
            .FirstOrDefaultAsync(uq => uq.UserId == userId && !uq.IsCompleted);

        if (userQuest == null) return BadRequest("No active quest to
complete.");

        // 2. Читаємо конфігурацію квесту для перевірки умов
        var filePath = Path.Combine(_env.ContentRootPath, "Quests", "npc_test",
${userQuest.QuestId}.json");
        if (!System.IO.File.Exists(filePath)) return NotFound("Quest file not
found.");

        var jsonString = await System.IO.File.ReadAllTextAsync(filePath);
        var jsonNode = JsonNode.Parse(jsonString);
        if (jsonNode == null) return BadRequest("Corrupt quest file.");

        var requiredAmount = jsonNode["objective"]?
["required_amount"]?.GetValue<int>() ?? 1;

```

```

// 3. Валідація виконання цілі
if (userQuest.CurrentAmount < requiredAmount)
{
    return BadRequest("Quest objectives are not completed yet.");
}

// 4. Помічаємо квест як виконаний
userQuest.IsCompleted = true;

// 5. Нараховуємо нагороду (досвід) гравцю
var xpReward = jsonNode["rewards"]?["xp"]?.GetValue<int>() ?? 0;
var user = await _context.Users.FindAsync(userId);
if (user != null)
{
    user.XP += xpReward;

    // Логіка підвищення рівня: Max XP = (Level + 1) * 100
    var neededXp = (user.Level + 1) * 100;
    while (user.XP >= neededXp)
    {
        user.XP -= neededXp;
        user.Level += 1;
        neededXp = (user.Level + 1) * 100;
    }
}

await _context.SaveChangesAsync();

return Ok(new
{
    message = "Quest completed successfully.",
    xpEarned = xpReward,
    newLevel = user?.Level,
    newXp = user?.XP
});
}
}

```

4. Специфіка взаємодії з клієнтом (Godot 4)

Бекендери мають знати:

1. **Авторизація:** Клієнт Godot передає JWT-токен у заголовок кожного запиту: `Authorization: Bearer <token>`. Контролер налаштований так, що автоматично перевіряє цей токен завдяки атрибуту `[Authorize]`.
2. **Формат повернення:** Всі методи повертають стандартний JSON. Для парсингу файлів конфігурацій квестів використовується сучасна бібліотека `.NET 8` — `System.Text.Json.Nodes.JsonNode`, що дозволяє змінювати поля JSON динамічно без опису складних `C#` класів під структуру квестів.